SOFTWARE ENGINEERING – LAB 2 – JIRA

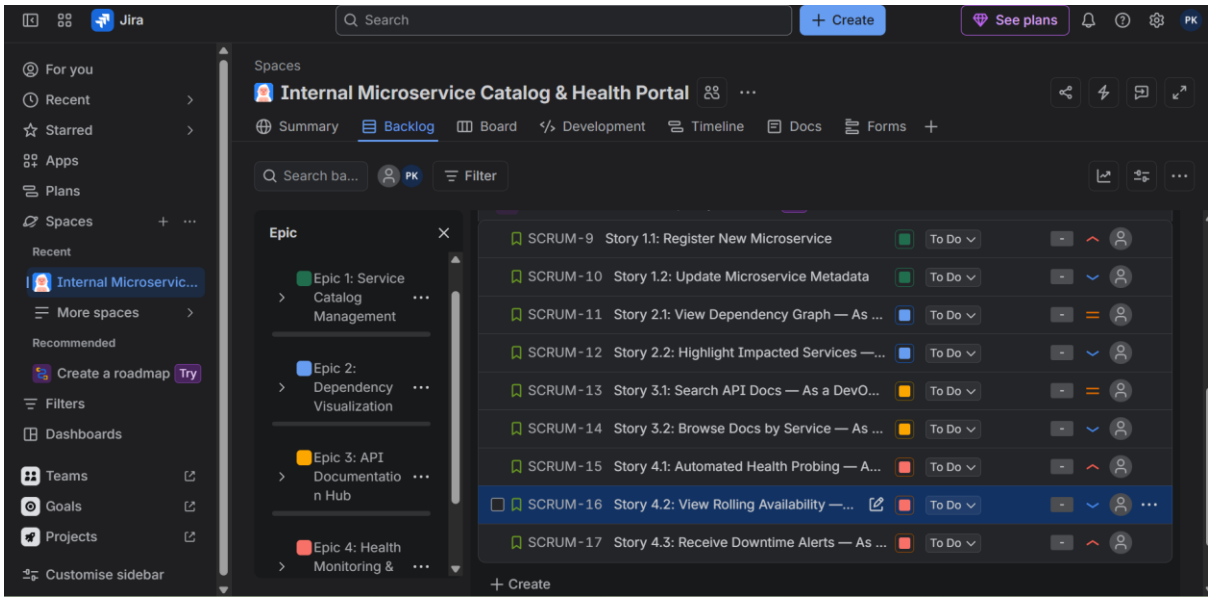
Name : Purav N Khincha

SRN: PES1UG24CS349

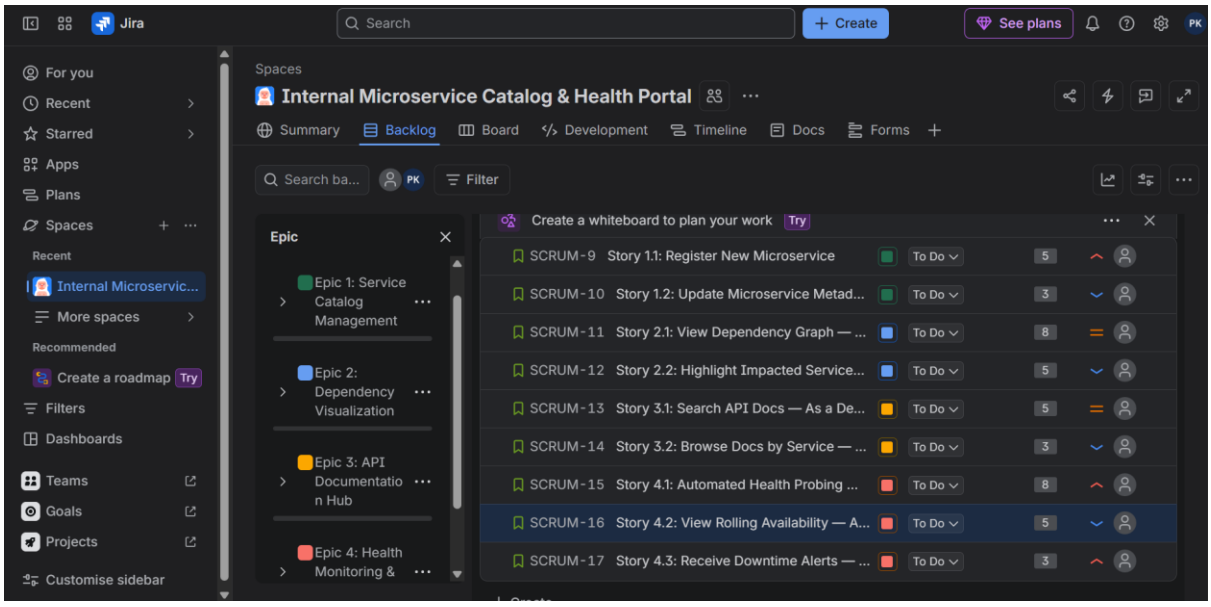
Section: 5F

Problem Statement: Internal Microservice Catalog & Health Portal

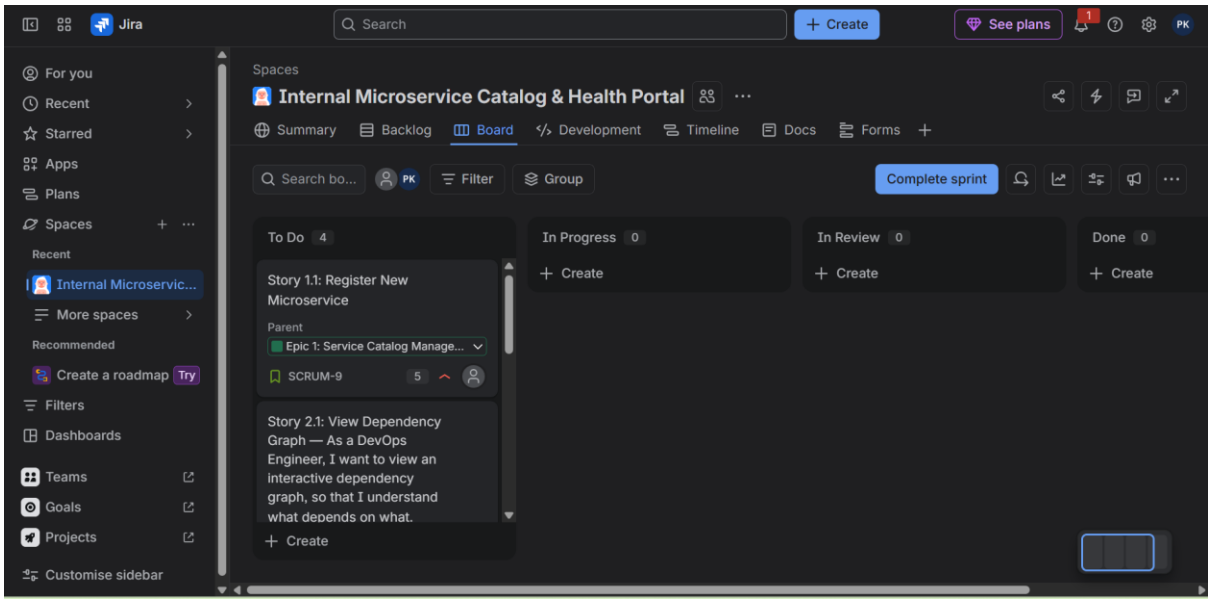
1) Assigning priorities to each story in the four different EPICS



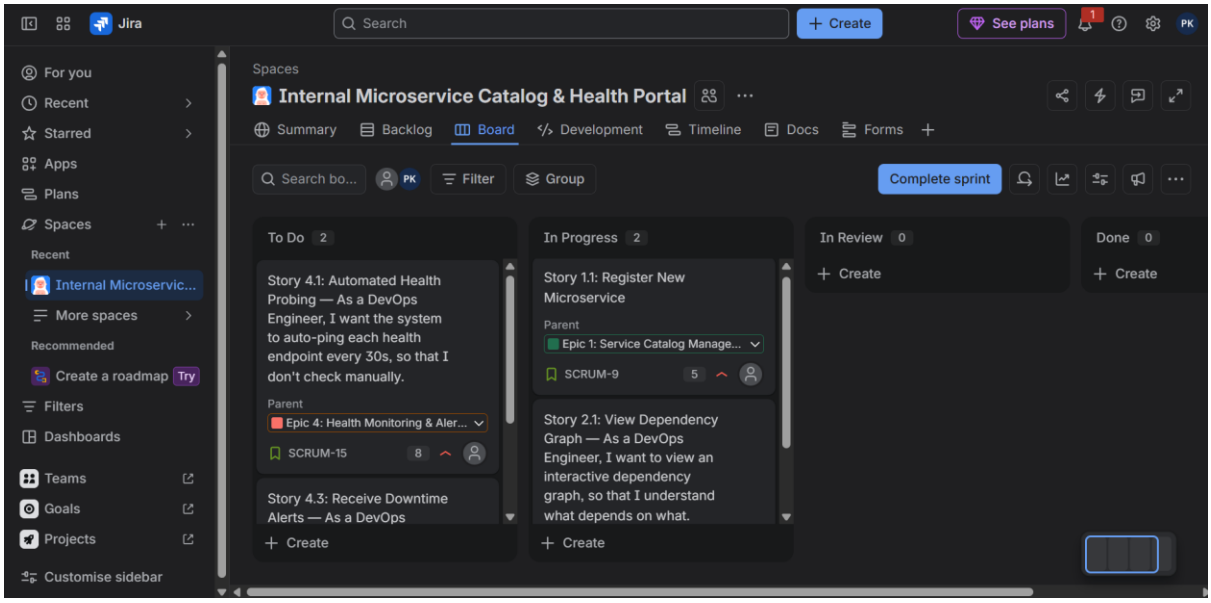
2) Assigning story point estimate to each story in all the four EPICS



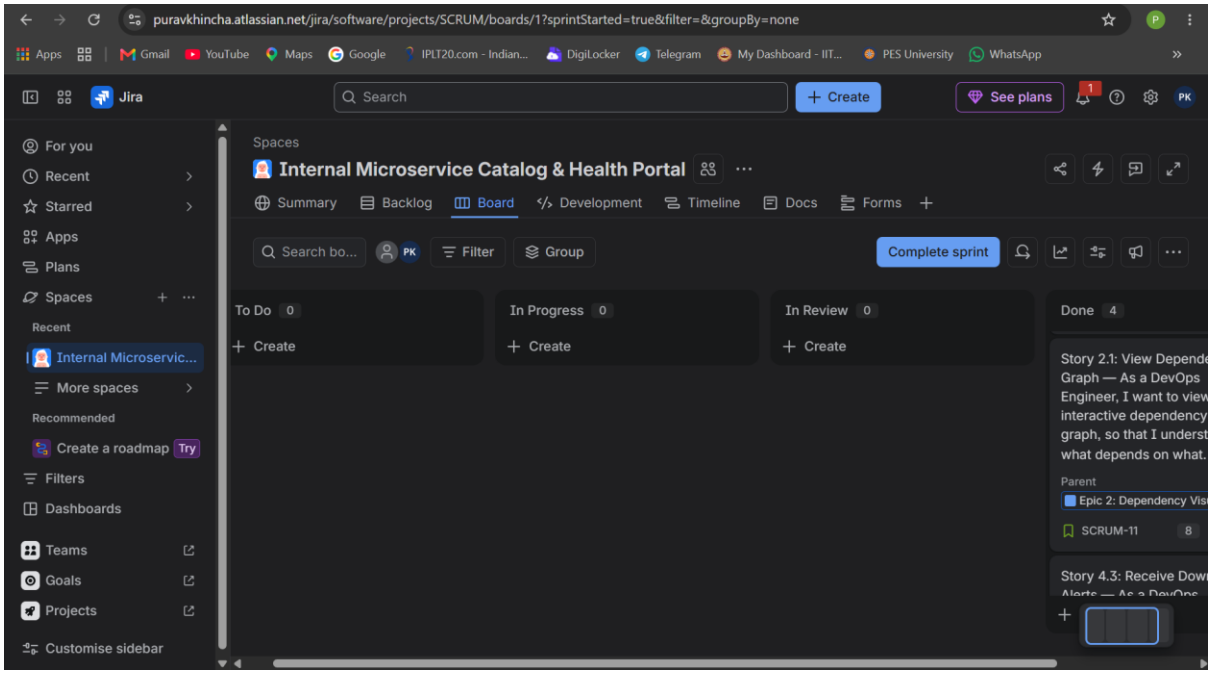
3) Starting Sprint 1



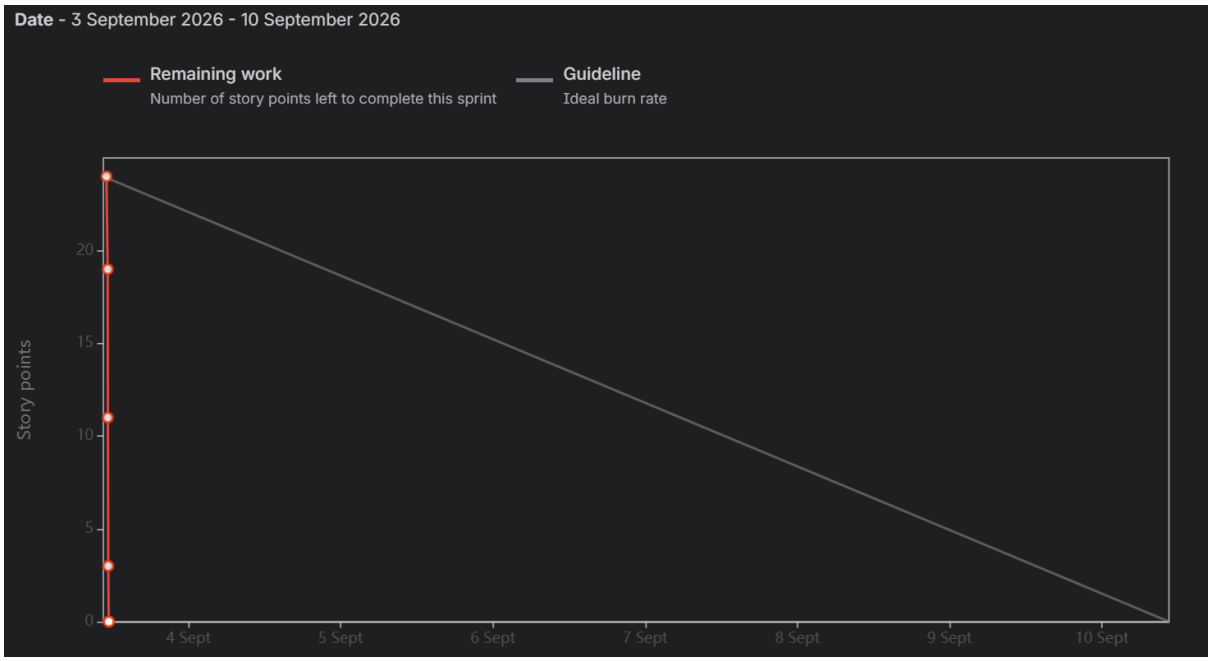
4) Mid Sprint 1



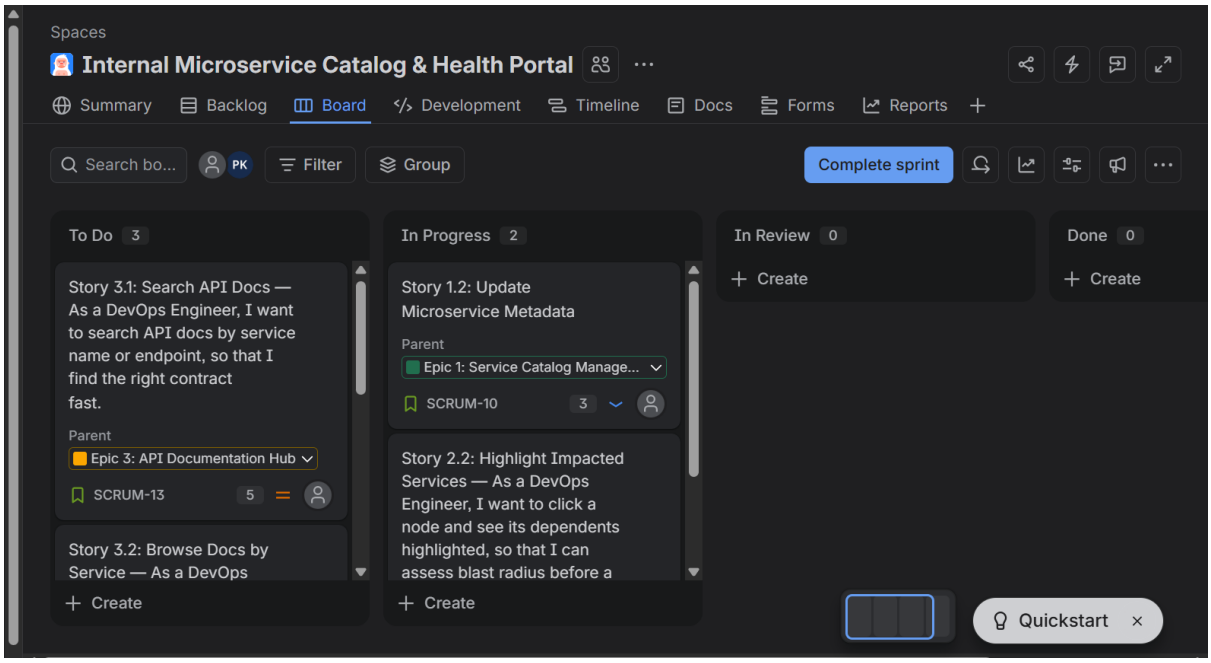
5) Sprint 1 complete



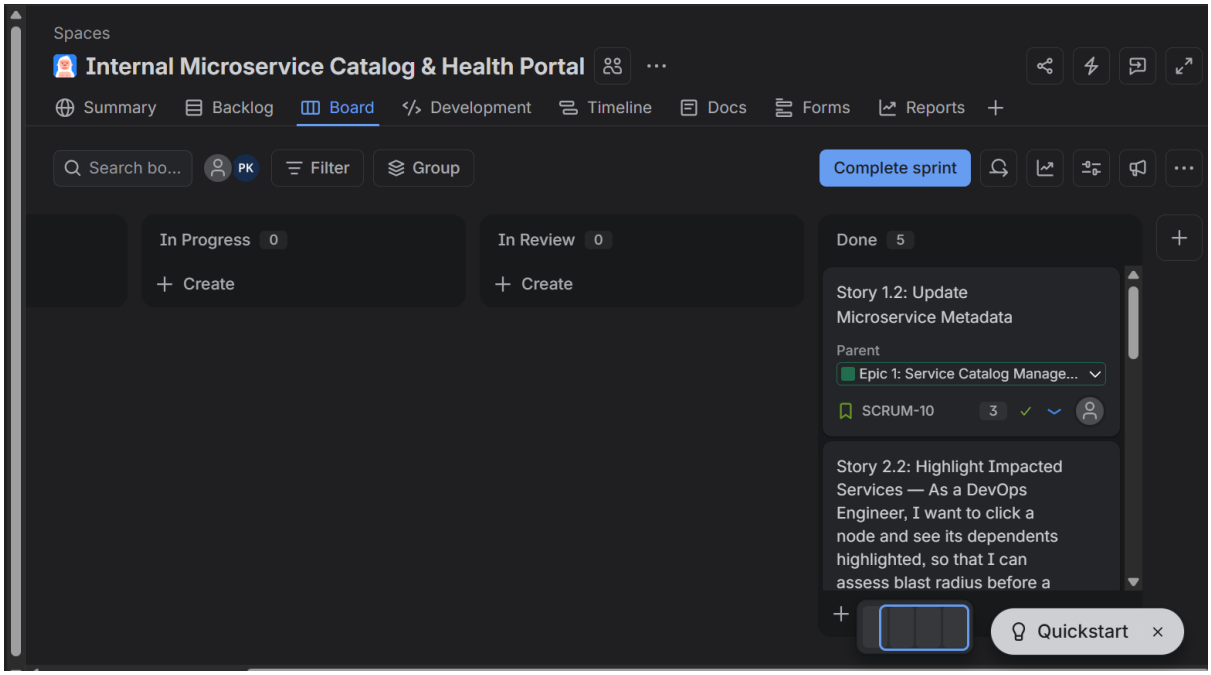
6) Burndown chart for Sprint 1



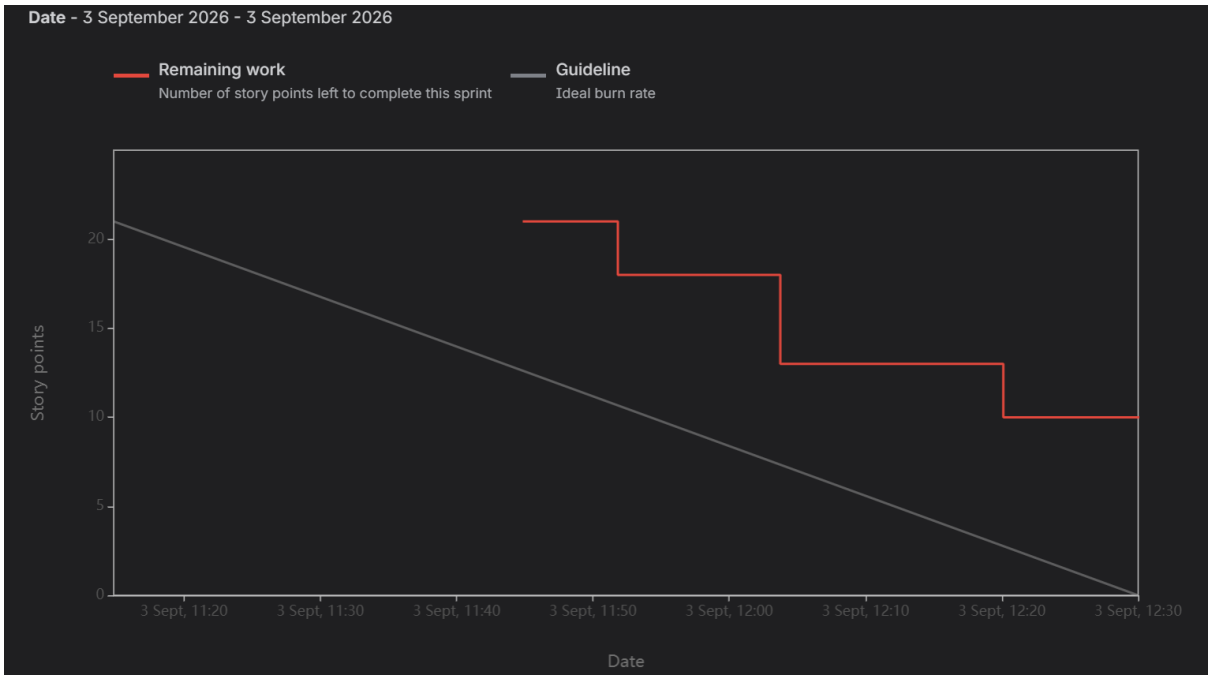
7) Mid Sprint 2



8) Sprint 2 complete



9) Burndown chart for Sprint 2



Reflection Questions :

1. Did your estimations reflect the actual effort?

Reasonably well for Sprint 1: all 4 stories (24 points total — Register New Microservice, View Dependency Graph, Automated Health Probing, Receive Downtime Alerts) were the highest-priority, highest-point items, and completing them first felt appropriate given they carry the core system logic. Sprint 2 is where estimation vs. reality diverged more visibly: the burndown shows all 21 points completed, but not within the scheduled 70-minute window I'd set — the last two 5-point stories (View Rolling Availability, Search API Docs) ran past the sprint's end time. That's a real signal that 21 points was over-committed for the window I'd planned, even though the story point values themselves (3s and 5s) were reasonable relative to each other.

2. Was your backlog well-prioritized?

Yes — the ordering held up in practice. Sprint 1 intentionally front-loaded the three High-priority stories (catalog registration, health probing, downtime alerts) plus one Medium (dependency graph), which are the capabilities the rest of the system depends on. Sprint 2 picked up the remaining Medium and Low items (metadata editing, doc browsing, availability view) — all genuinely lower-stakes refinements that could safely wait. Nothing in either sprint felt blocked by something left in the backlog, which suggests the priority levels were assigned correctly the first time.

3. How did your simulated sprint align with your plan?

Sprint 1 didn't align with its own plan at all — I'd set a 1-week sprint duration but finished all 4 stories in about 10 minutes, so the "plan" (a week of steady work) and the actual execution (near-instant) were completely mismatched. Sprint 2 aligned much better: I deliberately shortened the sprint window to match how long I actually intended to spend, and staggered

each story's completion by ~10 minutes rather than dumping them all at once. The visible step pattern (21→18→13→10) in the burndown is proof that this planning adjustment worked — though the final two stories still slipped past the scheduled end time, showing that even a shortened, more "realistic" plan can still under-estimate actual pace.

4. What insights did the burndown chart give about your team's capacity?

The two charts told very different stories. Sprint 1's chart showed a single vertical drop against a slowly-declining guideline — a textbook sign of a sprint window set far too long for the actual work done (the classic "team finishes early" pattern the lab materials describe, just in an extreme, compressed form). Sprint 2's stepped pattern is closer to how a real team's burndown should look, but it also revealed a subtler capacity issue: even after shortening the window, the last two stories still finished after the scheduled end date, meaning I'd still under-estimated how much real time 21 points of "work" — even simulated — would take. The lesson mirrors what the lab notes say about teams consistently missing their forecast: it's a sign of committing to slightly too much for the time box, not that the story points themselves were wrong.